

간식 분배 - 풀이

작성자: 박영우

부분문제 1

모든 학생은 정확히 하나의 간식을 좋아하므로, 학생이 방에 들어갔을 때 최대 하나의 간식을 가져간다.

만약 여러 학생이 공통으로 좋아하는 간식이 있다면, 그중 나중에 방에 들어가는 학생은 아무 간식도 가져갈 수 없다. 따라서 이 경우에는 목표를 달성할 수 없다.

그렇지 않다면 모든 학생이 서로 다른 간식을 좋아하므로, 학생들을 어떤 순서로 방에 들여보내더라도 모든 학생이 정확히 하나의 간식을 가져간다.

부분문제 2

이 부분문제에서는 목표를 달성하는 것이 가능할 경우, $1, 2, \dots, N$ 번 학생들을 이 순서대로 방에 들여보내도 목표를 달성할 수 있다.

따라서 방 안에 남은 간식을 관리하면서, 각 학생이 순서대로 방에 들어갈 때 가져가는 간식의 개수를 세면 된다.

부분문제 3

이 부분문제에서 학생들이 방에 들어가는 경우의 수는 최대 $N! \leq 120$ 가지이다. 따라서 각각의 경우에 대해 부분문제 2의 알고리즘을 사용하여 모든 학생이 정확히 하나의 간식을 가져가는지 판정하면 된다.

부분문제 4

$\{1, 2, \dots, N\}$ 의 모든 부분집합 S 에 대해, $dp[S]$ 를 다음과 같이 정의한다.

- 만약 S 에 속한 학생들의 순서를 잘 정하여, 그 순서대로 방에 들여보냈을 때 S 에 속한 각 학생이 정확히 하나의 간식을 가져가도록 할 수 있다면 $dp[S] = 1$ 이다.
- 그렇지 않다면 $dp[S] = 0$ 이다.

모든 S 에 대하여, $dp[S] = 1$ 인 조건은 다음과 같다.

- S 가 비어 있거나,
- S 에서 i 를 제거한 집합 T 가 있어서 $dp[T] = 1$ 이고, i 번 학생이 좋아하는 간식 중 T 에 속한 학생들이 좋아하는 간식이 아닌 것은 정확히 1개이다.

따라서, S 의 모든 부분집합 T 에 대해 $dp[T]$ 를 구했다면 $dp[S]$ 는 $\mathcal{O}(N)$ 에 구할 수 있다.

비트필드를 이용한 다이내믹 프로그래밍을 사용하면 $\mathcal{O}(N2^N)$ 에 풀 수 있다.

부분문제 5

현재 일부 학생들이 이미 방에 들어가 정확히 하나의 간식을 가져갔고, 남아 있는 간식 중 정확히 하나만 좋아하는 학생 i 가 있다고 하자.

현재 상태에서 남은 학생들의 순서를 정하여 모든 학생들이 정확히 하나의 간식을 가져가도록 할 수 있다고 하자. 그 순서에서 i 보다 먼저 들어오는 학생은 i 가 가져갈 유일한 간식을 좋아할 수 없다. 따라서 i 를 그 학생들보다 앞으로 옮겨도 다른 학생들이 가져가는 간식에 영향을 주지 않는다. 그러므로, 만약 현재 상태에서 목표를 달성하는 것이 가능하다면 i 를 바로 들여보내도 답이 존재한다.

따라서 다음과 같은 방법으로 문제를 풀 수 있다.

- 현재 남은 간식 중 정확히 하나를 좋아하는 학생이 없다면 불가능하다고 판정한다.
- 그렇지 않은 경우, 정확히 하나의 간식을 좋아하는 학생을 찾고, 그 학생을 방에 들여보낸다. 만약 그러한 학생이 여러 명일 경우, 어떤 학생을 먼저 들여보내더라도 상관없다.
- 위 과정을 모든 학생이 간식을 가져갈 때까지 반복한다.

부분문제 5의 경우, $N \leq 300$ 이다. 다음과 같이 구현할 수 있다.

- 학생과 간식의 관계를 $N \times N$ 배열로 저장한다. i 번 학생이 j 번 간식을 좋아한다면 i 행 j 열에 1을 저장한다.
- 아직 방에 들어가지 않은 모든 학생의 행을 확인하여, 현재 남은 선호 간식이 정확히 하나인 학생을 찾는다. 그러한 행이 존재하지 않는다면 불가능하다고 판정한다.
- 찾은 행을 i 번 행이라 하고, i 번 학생이 좋아하는 간식 중 현재 유일하게 남아있는 것을 j 번 간식이라 하자. 배열에서 j 열을 모두 지운다.
- 위 과정을 모든 학생이 방에 들어갈 때까지 총 N 번 반복한다.

한 단계에 $\mathcal{O}(N^2)$ 번의 연산이 사용되고, 총 N 번 반복해야 하므로 전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 6

부분문제 5의 알고리즘을 최적화할 수 있다. 각 학생 i 에 대해 i 번 학생이 좋아하는 간식 중 현재 방에 남은 것의 개수를 cnt_i 라 하고, 이 배열을 관리한다. 다음과 같이 해결할 수 있다.

- 방에 들어갈 학생을 구하는 것은 아직 방에 들어가지 않은 학생 중 $cnt_i = 1$ 인 학생 i 를 구하면 된다. 이때 i 번 학생이 방에 들어가면 정확히 하나의 간식 j 를 가져가게 된다.
- 이 과정에서 j 번 간식을 좋아하는 학생 k 에 대해, cnt_k 가 1 감소한다. j 번 간식을 좋아하는 모든 학생 k 를 찾아 cnt_k 를 갱신한다.

두 과정 모두 $\mathcal{O}(N)$ 에 처리할 수 있고, 총 N 번 반복하므로 전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 7

부분문제 6의 알고리즘에서 추가적인 최적화를 할 수 있다.

- $cnt_i \leq 1$ 인 학생들을 큐(queue) 등의 자료구조를 사용해 저장한다. 어떤 학생이 간식을 가져가서 학생 i 의 cnt_i 가 1 감소했다 하자. 이때 $cnt_i \leq 1$ 인지 검사하고, 만약 그렇다면 i 를 큐에 추가하면 된다.
- j 번 간식을 좋아하는 학생들을 구하는 것은, 처음에 간식 j ($1 \leq j \leq N$) 각각에 대해 j 번 간식을 좋아하는 학생들의 배열을 `std::vector` 등의 동적 배열을 만들어 사용하면 된다. 각 간식은 최대 한 번 제거되므로, 각 배열도 최대 한 번 확인된다.

전체 시간복잡도는 $\mathcal{O}(N + \sum C_i)$ 이다.

극솟값 제거 - 풀이

작성자: 장근영

풀이

각 쿼리 (l, r, t) 에서는 부분수열 $A_l, A_{l+1}, \dots, A_r$ 만을 따로 떼어 내고, 극솟값을 동시에 제거하는 과정을 t 번 적용한다.

양쪽 이웃이 모두 존재해야 제거될 수 있으므로, 현재 수열의 양 끝 원소는 절대로 제거되지 않는다.

부분문제 1

이 부분문제에서는 모든 쿼리가 전체 수열을 대상으로 하며, $N \leq 5000$ 이다.

현재 수열을 직접 저장하면서 각 변화를 그대로 시뮬레이션한다.

한 번의 변화를 처리할 때 현재 수열을 순회하여 극솟값을 모두 표시한다. 그 뒤 표시되지 않은 원소들만 순서대로 모아 새로운 수열을 만든다.

모든 극솟값은 동시에 제거되어야 하므로, 수열을 순회하는 도중에 원소를 바로 제거하면 안 된다는 점에 유의해야 한다.

s 번 변화시킨 뒤 남은 원소의 개수를 C_s 라고 저장하면, 쿼리 $(1, N, t)$ 의 답은 C_t 이다.

한 번의 변화에 $\mathcal{O}(N)$ 의 시간이 들고, 변화는 최대 N 번만 수행하면 된다.

전체 시간 복잡도는 $\mathcal{O}(N^2 + Q)$ 이다.

부분문제 2

이 부분문제에서도 모든 쿼리가 전체 수열을 대상으로 한다.

각 원소 A_i 가 전체 수열에서 몇 번째 변화에 제거되는지를 D_i 라고 하자. 끝까지 제거되지 않는 원소에 대해서는 $D_i = \infty$ 로 정의한다.

현재 살아 있는 원소들 사이의 인접 관계를 이중 연결 리스트로 관리한다.

처음에는 모든 극솟값을 후보 목록에 넣는다. 한 번의 변화에서는 후보 목록의 원소들을 모두 동시에 제거하고, 제거된 원소들의 기존 양쪽 이웃만 다시 극솟값인지 검사한다.

원소 하나가 제거되었을 때 극솟값 여부가 달라질 수 있는 원소는 기존의 양쪽 이웃뿐이다. 또한 각 원소는 최대한 한번만 제거된다.

따라서 모든 D_i 를 구하는 데 필요한 시간은 $\mathcal{O}(N)$ 이다.

각 시각까지 제거된 원소의 누적 개수를 저장하면 쿼리 $(1, N, t)$ 의 답은 $N - \#\{i \mid D_i \leq t\}$ 이다.

전체 시간 복잡도는 $\mathcal{O}(N + Q)$ 이다.

부분문제 3

이 부분문제에서는 항상 $t = 1$ 이다.

첫 번째 변화에서 위치 i 의 원소가 제거될 조건은

$$A_{i-1} > A_i < A_{i+1}$$

이다.

각 $2 \leq i < N$ 에 대해, i 가 극솟값이면 $C_i = 1$, 그렇지 않으면 $C_i = 0$ 으로 정의하고 C 의 누적 합을 구한다.

쿼리 $(l, r, 1)$ 에서 제거될 수 있는 위치는 양 끝을 제외한 $l + 1, l + 2, \dots, r - 1$ 뿐이다.

따라서 제거되는 원소 수는 $\sum_{i=l+1}^{r-1} C_i$ 이고, 답은

$$r - l + 1 - \sum_{i=l+1}^{r-1} C_i$$

이다.

전체 시간 복잡도는 $\mathcal{O}(N + Q)$ 이다.

부분문제 4

이 부분문제에서는 항상 $t = N$ 이다.

원소가 제거되는 변화에서는 적어도 하나의 원소가 없어지므로, N 번 변화시킨 뒤에는 반드시 더 이상 제거할 원소가 없다.

구간 $[l, r]$ 을 왼쪽에서 오른쪽으로 보았을 때, 지금까지 등장한 모든 원소보다 큰 원소를 접두사 최댓값이라고 하자.

마찬가지로 오른쪽에서 왼쪽으로 보았을 때, 지금까지 등장한 모든 원소보다 큰 원소를 접미사 최댓값이라고 하자.

접두사 최댓값은 자신의 왼쪽에 자신보다 큰 원소가 없다. 따라서 어떤 순간에도 왼쪽 이웃이 자신보다 커질 수 없으므로 극솟값이 될 수 없다.

접미사 최댓값도 같은 이유로 제거되지 않는다.

반대로 어떤 원소의 왼쪽과 오른쪽에 모두 자신보다 큰 원소가 존재한다고 하자.

값이 더 작은 원소부터 생각하면, 해당 원소와 양쪽의 더 큰 원소 사이에 있는 작은 원소들은 결국 모두 제거된다. 그러면 해당 원소의 양쪽 이웃이 모두 자신보다 커지므로 해당 원소도 제거된다.

따라서 최종적으로 남는 원소들은 정확히 접두사 최댓값이거나 접미사 최댓값인 원소들이다.

각 위치 i 에 대해 다음을 정의하자.

- L_i : i 의 왼쪽에서 가장 가까운, A_i 보다 큰 원소의 위치

- R_i : i 의 오른쪽에서 가장 가까운, A_i 보다 큰 원소의 위치

해당하는 원소가 없다면 각각 $L_i = 0, R_i = N + 1$ 로 둔다.

모든 L_i 와 R_i 는 단조 스택을 이용하여 $\mathcal{O}(N)$ 에 구할 수 있다.

구간 $[l, r]$ 의 접두사 최댓값들은 정확히

$$l, R_l, R_{R_l}, \dots$$

의 형태로 나타난다. 위치가 r 을 넘어가면 중단한다.

마찬가지로 접미사 최댓값들은

$$r, L_r, L_{L_r}, \dots$$

의 형태로 나타난다. 위치가 l 보다 작아지면 중단한다.

두 수열은 구간 $[l, r]$ 의 최댓값 위치에서 정확히 한 번 겹친다.

L/R 을 여러 번 적용하는 것을 빠르게 하기 위해서 아래와 같이 희소 배열을 만들어준다.

$$RJ_0(i) = R_i, \quad RJ_{k+1}(i) = RJ_k(RJ_k(i))$$

$RJ_k(i)$ 는 i 에서 R 을 2^k 번 적용한 결과이다.

마찬가지로

$$LJ_0(i) = L_i, \quad LJ_{k+1}(i) = LJ_k(LJ_k(i))$$

를 정의한다.

각 쿼리에 대해 희소 배열에서의 이분탐색을 사용하면 접두사 최댓값과 접미사 최댓값의 개수를 각각 $\mathcal{O}(\log N)$ 에 구할 수 있다.

두 개수를 더한 뒤, 구간의 최댓값이 두 번 세어졌으므로 1을 빼면 답을 얻는다.

전체 시간 복잡도는 $\mathcal{O}((N + Q) \log N)$ 이다.

부분문제 5

전체 수열은 어떤 위치 p 를 기준으로

$$A_1 > A_2 > \dots > A_p < A_{p+1} < \dots < A_N$$

의 형태이다.

따라서 어떤 구간을 선택하더라도 그 구간은 단조 수열이거나 V자 형태이다.

구간 $[l, r]$ 을 생각하자.

현재 수열에 극솟값이 존재한다면 극솟값은 정확히 하나이다. 이 원소를 제거한 뒤에도 남은 수열은 다시 V자 형태가 된다.

따라서 매 변화마다 원소가 정확히 하나씩 제거된다.

$M = \min(A_l, A_r)$ 라고 하자.

구간 내부에서 값이 M 보다 작은 원소가 남아 있다면, 그중 가장 작은 원소가 현재 수열의 극솟값이 되어 제거된다.

반면 값이 M 보다 작은 원소가 모두 제거되면, 값 M 을 가지는 양 끝 원소 중 하나가 남은 수열의 최솟값이다. 양 끝 원소는 제거되지 않으므로 이후에는 더 이상 극솟값이 존재하지 않는다.

따라서 전체 과정에서 제거되는 원소의 수는

$$K = \#\{i \mid l \leq i \leq r, A_i < M\}$$

이다.

매 변화마다 정확히 하나가 제거되므로, t 번 변화시킨 뒤 남은 원소의 개수는

$$r - l + 1 - \min(t, K)$$

이다.

수열은 p 의 왼쪽에서 단조 감소하고 오른쪽에서 단조 증가한다. 따라서 $A_i < M$ 을 만족하는 원소의 개수는 양쪽에서 각각 한 번씩 이분 탐색하여 구할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(N + Q \log N)$ 이다.

핵심 관찰

이제 일반적인 구간에 대해 여러 번의 변화를 처리하는 방법을 알아보자.

전체 수열에서의 제거 시각

각 위치 i 에 대해 D_i 를 전체 수열에서 A_i 가 제거되는 시각이라고 정의한다. 끝까지 제거되지 않는다면 $D_i = \infty$ 이다.

앞서 정의한 L_i 와 R_i 를 이용하면 모든 D_i 를 값이 작은 원소부터 차례대로 계산할 수 있다.

각 위치 i 에 대해 다음 집합을 정의하자.

$$C_i = \{x \mid L_x = i \text{ 또는 } R_x = i\}$$

$x \in C_i$ 이면 $A_x < A_i$ 이다.

또한 x 가 살아 있는 동안에는 i 의 한쪽에 i 보다 작은 원소가 남아 있으므로, i 는 x 보다 먼저 제거될 수 없다.

$L_i = 0$ 이거나 $R_i = N + 1$ 이면 i 의 한쪽에는 자신보다 큰 원소가 없다. 따라서 i 는 절대로 제거되지 않는다.

그 외의 경우에는 C_i 의 원소들이 모두 제거된 다음 변화에서 i 가 제거된다.

따라서 다음 관계가 성립한다.

$$D_i = \begin{cases} \infty, & L_i = 0 \text{ 또는 } R_i = N + 1, \\ 1 + \max_{x \in C_i} D_x, & \text{그 외.} \end{cases}$$

C_i 가 비어 있다면 최댓값은 0으로 정의한다.

모든 $x \in C_i$ 에 대해 $A_x < A_i$ 이므로, 원소들을 값이 작은 순서대로 처리하면 D_i 를 계산할 때 필요한 모든 값이 이미 계산되어 있다.

A 는 $1, 2, \dots, N$ 의 순열이므로 값이 v 인 원소의 위치를 미리 저장하면 모든 D_i 를 $\mathcal{O}(N)$ 에 계산할 수 있다.

구간에서도 제거되는 조건

위치 i 가 전체 수열에서 D_i 번째 변화에 제거된다고 해서, 임의의 구간에서도 같은 시각에 제거되는 것은 아니다.

예를 들어 $L_i < l$ 이라면, 구간 $[l, r]$ 안에는 i 의 왼쪽에 A_i 보다 큰 원소가 없다.

따라서 i 는 구간을 왼쪽에서 보았을 때의 접두사 최댓값이고, 구간 안에서는 절대로 제거되지 않는다.

마찬가지로 $R_i > r$ 이면 i 는 구간의 접미사 최댓값이므로 제거되지 않는다.

이제 $L_i \geq l$ 이고 $R_i \leq r$ 이라고 하자.

L_i 와 R_i 는 모두 구간 안에 있으며 값이 A_i 보다 크다. 또한 i 가 제거되기 전까지는 두 원소 모두 제거되지 않는다.

따라서 L_i 와 R_i 사이에서 일어나는 제거 과정은 그 바깥 원소들의 영향을 받지 않는다.

그러므로 구간 $[l, r]$ 에서도 i 는 전체 수열에서와 같은 시각 D_i 에 제거된다.

결론적으로 위치 i 가 쿼리 (l, r, t) 에서 제거될 필요충분조건은 다음과 같다.

$$D_i \leq t, \quad L_i \geq l, \quad R_i \leq r$$

구간의 경계에서 살아남는 원소

먼저 구간 $[l, r]$ 안에서 $D_i \leq t$ 인 원소를 모두 세었다고 하자.

그중 실제 구간에서는 제거되지 않는 원소는 $L_i < l$ 이거나 $R_i > r$ 인 원소이다.

$L_i < l$ 인 원소들은 구간의 접두사 최댓값들이며, 정확히

$$l, R_l, R_{R_l}, \dots$$

의 형태로 나타난다.

$R_i > r$ 인 원소들은 구간의 접미사 최댓값들이며, 정확히

$$r, L_r, L_{L_r}, \dots$$

의 형태로 나타난다.

이 두 체인은 구간의 최댓값 위치에서만 접친다.

따라서 실제로 제거되는 원소의 수는 다음과 같이 구할 수 있다.

1. 구간 안에서 $D_i \leq t$ 인 원소를 모두 센다.
2. 왼쪽 끝에서 시작하는 R 체인 중 $D_i \leq t$ 인 원소를 뺀다.
3. 오른쪽 끝에서 시작하는 L 체인 중 $D_i \leq t$ 인 원소를 뺀다.
4. 구간의 최댓값을 두 번 뺐다면 한 번 다시 더한다.

체인을 따라가면 제거 시각이 증가한다

R_i 가 존재한다고 하자.

R_i 의 정의에 의해 R_i 는 i 의 오른쪽에서 가장 가까운, A_i 보다 큰 원소이다. 따라서 R_i 의 제거 시각을 계산할 때 i 는 C_{R_i} 에 포함된다.

그러므로 D_i 가 유한하다면

$$D_{R_i} \geq D_i + 1$$

이다.

같은 이유로 L_i 가 존재하고 D_i 가 유한하다면

$$D_{L_i} \geq D_i + 1$$

이다.

따라서 R 을 반복해서 적용한 체인에서는 제거 시각이 엄격히 증가한다.

$$D_i < D_{R_i} < D_{R_{R_i}} < \dots$$

마찬가지로 L 을 반복해서 적용한 체인에서도 제거 시각이 엄격히 증가한다.

$$D_i < D_{L_i} < D_{L_{L_i}} < \dots$$

따라서 각 체인에서 $D_i \leq t$ 인 원소들은 항상 체인의 앞쪽 연속 구간을 이룬다.

이 성질 덕분에 체인의 모든 원소를 하나씩 확인하지 않고, 이분 탐색으로 조건을 만족하는 마지막 원소를 찾을 수 있다.

부분문제 6

전체 수열에 과정을 20번 적용한 뒤 더 이상 제거되는 원소가 없다.

따라서 모든 유한한 제거 시각은 $D_i \leq 20$ 을 만족한다.

각 $1 \leq s \leq 20$ 에 대해 다음 누적 합을 만든다.

$$P_s(i) = \#\{j \mid 1 \leq j \leq i, D_j \leq s\}$$

쿼리 (l, r, t) 에서 전체 수열을 기준으로 t 번 이내에 제거되는 원소의 수는

$$P_{\min(t, 20)}(r) - P_{\min(t, 20)}(l - 1)$$

이다.

이제 왼쪽 끝 l 에서 R 을 반복해서 적용하면서, 위치가 r 이하이고 제거 시각이 t 이하인 원소들을 뺀다.

체인에서 제거 시각은 엄격히 증가하며 유한한 제거 시각은 최대 20이므로, 최대 20개의 원소만 확인하면 된다.

오른쪽 끝 r 에서도 L 을 반복해서 적용하여 같은 작업을 한다.

두 체인이 같은 원소까지 도달했다면 그 원소를 두 번 뺀 것이므로 한 번 다시 더한다.

쿼리 하나를 $\mathcal{O}(20)$ 에 처리할 수 있다.

전체 시간 복잡도는 $\mathcal{O}(20N + 20Q)$ 이고, 메모리 복잡도는 $\mathcal{O}(20N)$ 이다.

부분문제 7

일반적인 경우에는 체인의 길이가 클 수 있으므로 이분 탐색을 사용한다.

각 위치 i 에 대해 시각 D_i 에 위치 i 를 활성화한다고 생각하자.

쿼리들도 t 가 작은 순서대로 처리한다.

현재 처리하는 쿼리의 시간이 t 라면, $D_i \leq t$ 인 모든 위치 i 를 Fenwick Tree에 추가한다.

그러면 구간 $[l, r]$ 에서 활성화된 원소의 수

$$C = \#\{i \mid l \leq i \leq r, D_i \leq t\}$$

를 Fenwick Tree의 구간 합으로 구할 수 있다.

위치의 활성화와 쿼리 처리를 모두 시간순으로 정렬하면 전체 시간 복잡도는 $\mathcal{O}((N + Q) \log N)$ 이다.

왼쪽 체인과 오른쪽 체인 위의 위치 i 중 $D_i \leq t$ 를 만족하는 i 의 개수를 세는 것은 희소 배열에서의 이분탐색을 사용하면 된다.

전처리에는 $\mathcal{O}(N \log N)$ 이 필요하고, 각 쿼리는 $\mathcal{O}(\log N)$ 에 처리된다.

전체 시간 복잡도는 $\mathcal{O}((N + Q) \log N)$ 이고, 메모리 복잡도는 $\mathcal{O}(N \log N)$ 이다.

다른 만점 풀이 1: CDQ 분할 정복

위치 i 가 쿼리 (l, r, t) 에서 제거될 필요충분조건은

$$L_i \geq l, \quad R_i \leq r, \quad D_i \leq t$$

이다.

첫 번째 부등식의 방향을 바꾸기 위해 $X_i = N - L_i + 1$ 로 정의하자.

그러면 $L_i \geq l$ 은 $X_i \leq N - l + 1$ 과 동치이다.

각 원소 i 를 다음 3차원 점으로 생각한다.

$$(N - L_i + 1, R_i, D_i)$$

쿼리 (l, r, t) 는 세 좌표가 모두

$$(N - l + 1, r, t)$$

이하인 점의 개수를 묻는다.

따라서 문제는 3차원 dominance counting 문제가 된다.

점과 쿼리를 첫 번째 좌표 순서대로 정렬하고 CDQ 분할 정복을 수행한다.

각 분할 정복 단계에서는 왼쪽 절반에 있는 점이 오른쪽 절반에 있는 쿼리에 주는 기여를 계산한다.

왼쪽의 점과 오른쪽의 쿼리를 두 번째 좌표 순서로 처리하면서, 두 번째 좌표가 현재 쿼리 이하인 점들을 Fenwick Tree에 추가한다.

Fenwick Tree에서는 세 번째 좌표를 관리한다. 그러면 세 좌표가 모두 쿼리 이하인 점의 수를 셀 수 있다.

센 점의 개수는 제거되는 원소의 수이므로, 구간의 길이에서 이 값을 빼면 답을 얻는다.

각 분할 정복 단계에서 정렬을 수행하면 전체 시간 복잡도는 $\mathcal{O}((N + Q) \log^2(N + Q))$ 이다.

다른 만점 풀이 2: 2차원 Fenwick Tree

각 원소 i 를 다음 2차원 점으로 생각하자.

$$x_i = N - L_i + 1, \quad y_i = R_i$$

점들을 D_i 순서로 정렬하고, 쿼리들을 t 순서로 정렬한다.

쿼리를 시간순으로 처리하면서 $D_i \leq t$ 가 된 점 (x_i, y_i) 를 2차원 Fenwick Tree에 추가한다.

쿼리 (l, r, t) 에서는

$$x_i \leq N - l + 1, \quad y_i \leq r$$

인 점의 개수를 세면 된다.

즉, 원점부터 $(N - l + 1, r)$ 까지의 직사각형 안에 있는 점의 개수를 구하면 된다.

일반적인 $N \times N$ 크기의 배열을 사용하면 메모리가 너무 크다.

대신 바깥쪽 Fenwick Tree의 각 정점에 실제로 추가될 수 있는 y 좌표들만 미리 모아 각각 좌표 압축한다.

점 하나는 바깥쪽 Fenwick Tree의 $\mathcal{O}(\log N)$ 개 정점에 포함되므로, 저장되는 좌표의 총개수는 $\mathcal{O}(N \log N)$ 이다.

점 추가와 직사각형 합 쿼리는 모두 $\mathcal{O}(\log^2 N)$ 에 처리할 수 있다.

전체 시간 복잡도는 $\mathcal{O}((N + Q) \log^2 N)$ 이고, 메모리 복잡도는 $\mathcal{O}(N \log N)$ 이다.

곡예 - 풀이

작성자: 송준혁

곡예사 Alice가 l 번 칸, Bob이 r 번 칸에서 있는 상황을 상태 (l, r) 라고 하자. 문제의 조건에 의해 모든 순간 $1 \leq l < r \leq N$ 이 성립한다.

부분문제 1

상태 (l, r) 를 정점으로 하는 그래프를 생각하여 너비 우선 탐색을 수행할 수 있다. 각 상태 (l, r) 에서,

- Alice의 걷기로 상태 $(l+1, r)$ 로, Bob의 걷기로 상태 $(l, r-1)$ 로 이동할 수 있다.
- l 번 칸과 r 번 칸에 설치된 각 점프대에 대해, 점프 후의 상태로 이동할 수 있다.

물론 조건을 위반하는 전이는 제외해야 한다. 질의마다 시작 상태에서 DFS 등의 그래프 탐색을 수행해 목표 상태의 도달 가능 여부를 확인하면 된다. 상태는 $O(N^2)$ 개, 간선은 $O(N^2 + NM)$ 개이므로 전체 시간복잡도는 $O(Q(N^2 + NM))$ 이고, $N, M, Q \leq 100$ 이므로 충분하다.

부분문제 2

먼저 걷기의 성질을 살펴보자. 상태 (x, y) 에서 Bob이 y' 번 칸까지 왼쪽으로 걸은 뒤 Alice가 x' 번 칸까지 오른쪽으로 걸으면, $x \leq x' < y' \leq y$ 를 만족하는 임의의 상태 (x', y') 에 도달할 수 있고, 이 과정의 모든 중간 상태가 조건을 만족한다. 즉, 구간을 좁히는 것은 자유롭다.

반대로 구간을 넓히는 이동을 생각하자. 상태 (x, y) 에서,

- $x \leq p < y$ 인 칸 p 에 $q < x$ 번 칸으로 향하는 점프대가 있다면, Alice가 p 번 칸까지 걸어가 점프하는 것으로 상태 (q, y) 에 도달할 수 있다.
- $x < p \leq y$ 인 칸 p 에 $q > y$ 번 칸으로 향하는 점프대가 있다면, Bob이 p 번 칸까지 걸어가 점프하는 것으로 상태 (x, q) 에 도달할 수 있다.

이러한 이동을 확장 이동이라고 하고, 확장 이동이 불가능한 상태를 닫힌 상태라고 하자.

관찰 1. 상태 (l, r) 에서 도달 가능한 가장 넓은 구간 (l^*, r^*) 가 유일하게 존재하고, 그 안에 포함되는 상태가 (l, r) 에서 도달 가능한 모든 상태이다. 즉, (l, r) 에서 (x, y) 에 도달할 수 있을 필요충분조건은 $l^* \leq x < y \leq r^*$ 이다.

관찰 2. 어떤 상태에서도, 매 순간 두 곡예사 사이의 점프대 중 구간을 넓힐 수 있는 것을 골라 확장 이동을 반복하면 어떤 순서로 반복하더라도 가장 넓은 구간 (l^*, r^*) 에 도달한다.

증명. 닫힌 상태 (X, Y) 와 $[x, y] \subseteq [X, Y]$ 인 상태 (x, y) 를 생각하자. 핵심은 (x, y) 에서 어떤 행동을 하더라도 구간이 $[X, Y]$ 를 벗어나지 못한다는 것이다. 걷기는 구간을 좁히므로 자명하다. Alice의 점프는 $[X, Y-1]$ 에 속한 칸에서 일어나는데, (X, Y) 가 닫힌 상태이므로 이 칸들에는 X 보다 왼쪽으로 향하는 점프대가 없다. 마찬가지로 Bob의 점프는 $[X+1, Y]$ 에 속한 칸에서 일어나므로 Y 보다 오른쪽에 착지할 수 없다.

이제 (l, r) 에서 확장 이동을 반복하자. 확장 이동은 구간을 항상 넓히고 구간은 $[1, N]$ 을 넘을 수 없으므로, 어떤 순서로 반복하더라도 유한 번 뒤에 닫힌 상태 (L, R) 에 도달한다. 앞 문단에 의해 $[l, r]$ 을 포함하는 임의의 닫힌 상태 (X, Y) 에 대해 확장 과정 전체가 $[X, Y]$ 안에 머무르므로 $[L, R] \subseteq [X, Y]$ 이다. 즉 (L, R) 은 $[l, r]$ 을 포함하는 포함 관계상 가장 작은 닫힌 상태이고, 확장 순서와 무관하게 유일하다. 이를 (l^*, r^*) 라고 하자.

(l^*, r^*) 에 도달한 뒤 걷기로 두 꼭짓점 사이의 거리를 좁히면 $l^* \leq x < y \leq r^*$ 인 모든 상태 (x, y) 에 도달할 수 있다. 반대로 (l^*, r^*) 는 $[l, r]$ 을 포함하는 닫힌 상태이므로, 증명의 첫 문단에 의해 어떤 행동의 나열로도 구간이 $[l^*, r^*]$ 를 벗어날 수 없다. 따라서 도달 가능한 상태의 집합은 정확히 $\{(x, y) : l^* \leq x < y \leq r^*\}$ 이고, (l^*, r^*) 가 도달 가능한 유일한 가장 넓은 구간이다. ■

따라서 질의 (a, b, c, d) 의 답은 (a, b) 의 가장 넓은 구간 (l^*, r^*) 에 대해 $l^* \leq c$ 이고 $d \leq r^*$ 인지 확인하는 것으로 판정할 수 있다.

이제 부분문제의 조건을 사용하자. 왼쪽으로 향하는 점프대가 단 하나도 없다는 조건에 의해 상태 (l, r) 에서 출발한 모든 상태 (x, y) 에 대해 $l \leq x$ 가 성립한다. 따라서 $l^* = a$ 이고, 답은 $a \leq c$ 이고 $d \leq r^*$ 인지로 판정된다. 즉, Alice의 이동은 고려할 필요 없이 Bob의 이동 범위만을 고려한다. 여기서 r^* 는 a 가 작아질수록 단조하게 커진다는 점에 주목하자. Bob이 내려가 사용할 수 있는 칸이 늘어나기 때문이다.

질의를 a 가 큰 것부터 오프라인으로 쿼리를 처리하자. 칸 p 에 설치된 점프대의 착지 칸들과 p 중 최댓값을 $g(p)$ 라고 하자. $a = i$ 에 도달하면 칸 $i + 1$ 이 Bob이 설 수 있는 칸으로 새로 추가된다. Alice가 i 위치에 고정되어 있을 때 Bob이 양방향으로 도달 가능한 칸들의 연속 구간을 관리하자. 칸 $i + 1$ 이 추가될 때 구간 $[i + 1, g(i + 1)]$ 과 만나는 컴포넌트들을 왼쪽부터 차례대로 합치면 되는데, 각 기존 컴포넌트는 이미 자신의 점프대들에 대해 닫혀 있으므로 합쳐진 컴포넌트도 닫혀 있다. 질의 (a, b, c, d) 는 i 가 a 에 도달했을 때, 칸 b 가 속한 컴포넌트의 가장 오른쪽 위치를 r^* 로 하여 판정한다.

시간복잡도는 $O((N + M + Q) \alpha(N))$ 이다.

부분문제 3

관찰 1에 의해, 모든 상태 (a, b) 에 대해 가장 넓은 구간 (l^*, r^*) 를 전처리해 두면 각 질의를 $O(1)$ 에 판정할 수 있다. 부분 문제 1의 그래프 탐색 풀이와 같이 상태 (l, r) 을 정점으로 하되 간선을 다음 4개로 줄이자.

- 걷기: 상태 $(l + 1, r)$ 과 $(l, r - 1)$ 로의 간선.
- Alice의 점프: l 번 칸의 점프대 중 가장 왼쪽 착지 칸으로의 간선.
- Bob의 점프: r 번 칸의 점프대 중 가장 오른쪽 착지 칸으로의 간선.

나머지 점프를 지워도 도달 가능성은 변하지 않는다. 구간을 좁히는 점프는 걷기로 대체할 수 있고, 가장 멀리 가지 않는 점프는 가장 먼 점프 후 걷기로 대체할 수 있기 때문이다.

이 그래프는 좁혔다가 점프로 되돌아오는 등의 사이클을 가질 수 있으므로, Kosaraju's algorithm이나 Tarjan's algorithm을 활용해 강한 연결 요소로 압축하자. 같은 강한 연결 요소에 속한 상태들은 서로 도달 가능하므로 같은 (l^*, r^*) 를 가진다. 압축된 DAG의 위상 정렬 역순으로, 각 컴포넌트에 속한 상태들의 l 의 최솟값과 r 의 최댓값에서 시작해 나가는 간선 너머의 값과 합치며 전파하면 모든 상태의 (l^*, r^*) 를 얻는다.

시간복잡도는 $O(N^2 + M + Q)$ 이다.

부분문제 4

각 질의마다 (a, b) 에서 확장 이동을 반복해 (l^*, r^*) 를 직접 구하자. 관찰 2에 의해 확장은 어떤 순서로 수행해도 같은 곳에 도달하므로, 양쪽을 한꺼번에 최대한 넓히는 것을 반복하면 된다.

칸 p 에 설치된 점프대의 착지 칸들과 p 중 최솟값을 $f(p)$, 최댓값을 $g(p)$ 라고 하자. 현재 상태 (x, y) 에서 한 번의 확장은

$$x \leftarrow \min_{x \leq p \leq y-1} f(p), \quad y \leftarrow \max_{x+1 \leq p \leq y} g(p)$$

이고, 이는 구간 최솟값/최댓값 질의(RMQ) 두 번이므로 세그먼트 트리를 이용해 $O(\log N)$ 또는 Sparse table을 이용해 $O(1)$ 에 수행할 수 있다. 반복할 때마다 구간이 1 이상 넓어지거나 더 이상 변하지 않으므로, 반복은 질의당 $O(N)$ 번이다. 반복이 멈춘 상태가 곧 확장 이동이 불가능한 닫힌 상태, (l^*, r^*) 이다.

전처리 $O(N \log N + M)$, 질의당 $O(N)$ 또는 $O(N \log N)$ 으로 전체 시간복잡도는 $O(N \log N + M + QN)$ 이고, $Q \leq 10$ 이므로 충분하다.

부분문제 5

관찰 1에 의해 모든 상태 (a, b) 의 가장 넓은 구간을 전처리해 두면 각 질의를 $O(1)$ 에 판정할 수 있지만, 상태는 $O(N^2)$ 개이므로 전부 전처리할 수는 없다. 대신 두 꼭예사가 인접해 있는 상태 $(i, i+1)$ 에 대응하는 단위 구간 $[i, i+1]$ ($1 \leq i \leq N-1$)들만 전처리하자. 상태 $(i, i+1)$ 의 가장 넓은 구간을 $[L^*(i), R^*(i)]$ 라고 하자.

핵심 관찰은, 큰 구간의 최대 닫힌 구간이 그 안에 포함된 단위 구간들의 도달 가능한 최대 닫힌 구간들의 합집합으로 표현된다는 것이다.

관찰 3. 상태 (l, r) 의 가장 넓은 구간 $[l^*, r^*]$ 에 대해 다음이 성립한다.

$$[l^*, r^*] = \bigcup_{i=l}^{r-1} [L^*(i), R^*(i)]$$

증명. 우변을 U 라고 하자. U 의 각 항이 (l, r) 에서 도달 가능한 영역임을 보여 $U \subseteq [l^*, r^*]$ 를 얻고, U 가 닫힌 상태임을 보여 반대 방향의 포함을 얻는 것으로 등식을 증명하자.

($\supseteq$) 겹기로 구간을 좁히는 것은 자유로우므로, (l, r) 에서 각 상태 $(i, i+1)$ ($l \leq i \leq r-1$)에 도달할 수 있고, $(i, i+1)$ 에서 도달 가능한 상태는 모두 (l, r) 에서도 도달 가능하다. 따라서 관찰 1에 의해 $[L^*(i), R^*(i)] \subseteq [l^*, r^*]$ 이다.

($\subseteq$) $U = [X, Y]$ 가 닫힌 상태임을 보이면, 관찰 2의 증명에서 $[l^*, r^*]$ 가 $[l, r]$ 을 포함하는 가장 작은 닫힌 상태였으므로 $[l^*, r^*] \subseteq U$ 가 따라온다. 확장 이동이 가능하다고 가정하자. 예컨대 어떤 칸 p ($X \leq p < Y$)에 $q < X$ 번 칸으로 향하는 점프대가 있다고 하자. $[p, p+1] \subseteq [L^*(i), R^*(i)]$ 인 항 i 가 존재하므로, 관찰 1에 의해 상태 $(p, p+1)$ 은 상태 $(i, i+1)$ 에서 도달 가능하다. 그런데 상태 $(p, p+1)$ 에서 Alice가 이 점프대로 점프하면 상태 $(q, p+1)$ 에 도달하므로, 다시 관찰 1에 의해 $q \geq L^*(i) \geq X$ 이다. 이는 모순이므로 왼쪽으로의 확장 이동은 불가능하고, 오른쪽도 대칭적이다. ■

관찰 3에 의해 질의 (a, b, c, d) 는

$$l^* = \min_{a \leq i \leq b-1} L^*(i), \quad r^* = \max_{a \leq i \leq b-1} R^*(i)$$

를 RMQ 두 번으로 구한 뒤 $l^* \leq c$ 이고 $d \leq r^*$ 인지 확인하는 것으로 판정된다. 남은 것은 L^* 와 R^* 의 계산이다.

Doubling을 이용한 로그 제곱 풀이. 부분문제 4의 확장을 상태 $(i, i+1)$ 에서 2^t 번 반복해 얻는 구간을 $[L_t(i), R_t(i)]$ 라고 하자. 한 번의 확장은 $[f(i), g(i+1)]$ 이므로 $L_0(i) = f(i)$, $R_0(i) = g(i+1)$ 이다.

확장 한 번은 정의상, 현재 구간에 포함된 각 단위 구간을 한 번씩 확장한 것들의 합집합이다.

$$\left[\min_{x \leq p \leq y-1} f(p), \max_{x+1 \leq p \leq y} g(p) \right] = \bigcup_{p=x}^{y-1} [f(p), g(p+1)]$$

합집합에 포함된 단위 구간은 어떤 한 항에 통째로 포함되므로 항별로 묶을 수 있고, 이 분해는 확장을 반복해도 유지된다. 즉 구간 $[x, y]$ 에서 확장을 k 번 반복한 구간은, 각 단위 구간 $[p, p+1]$ ($x \leq p \leq y-1$)에서 확장을 k 번 반복한 구간들의 합집합이다. 따라서 확장을 2^{t+1} 번 반복하는 것은 먼저 2^t 번 반복해 $[L_t(i), R_t(i)]$ 를 얻은 뒤, 그 안의 각 단위 구간에서 다시 2^t 번씩 반복한 것들을 합치는 것과 같다.

$$[L_{t+1}(i), R_{t+1}(i)] = \bigcup_{p=L_t(i)}^{R_t(i)-1} [L_t(p), R_t(p)]$$

즉 $L_{t+1}(i)$ 와 $R_{t+1}(i)$ 는 각각 배열 L_t, R_t 위의 구간 최솟값/최댓값이고, 한 라운드는 RMQ를 $O(N)$ 번 수행하는 것으로 $O(N \log N)$ 에 계산할 수 있다. 확장은 한 번에 구간을 1 이상 넓히거나 고정점에서 멈추므로 N 번 반복이면 충분하고, 따라서 $\lceil \log_2 N \rceil$ 라운드 뒤에는 모든 i 에 대해 $[L_t(i), R_t(i)] = [L^*(i), R^*(i)]$ 이다.

전체 시간복잡도는 $O(N \log^2 N + M + Q)$ 이다.

세그먼트 트리 간선과 SCC를 이용한 로그 풀이. Doubling을 이용하는 대신, 단위 구간들을 정점으로 하는 그래프에서 L^* 와 R^* 를 직접 구할 수도 있다. 관찰 1에 의해 상태 $(i, i+1)$ 에서 상태 $(j, j+1)$ 에 도달할 수 있을 필요충분조건은 $[j, j+1] \subseteq [L^*(i), R^*(i)]$ 이다. 한 번의 확장으로 얻는 구간 $[f(i), g(i+1)]$ 에 포함된 단위 구간들로 가는 간선 $i \rightarrow j$ ($f(i) \leq j \leq g(i+1) - 1$)를 연결하자. 간선은 관찰 1에 의해 실제로 도달 가능한 상태로만 이어지고, 역으로 확장을 반복해 구간이 넓어지는 과정을 따라가면, 현재 구간에 포함된 단위 구간이 모두 그래프에서 도달 가능하다는 사실이 귀납적으로 유지된다. 즉 이 그래프에서 i 로부터 도달 가능한 정점의 집합이 정확히 $[L^*(i), R^*(i)]$ 에 포함된 단위 구간들이므로, 도달 가능한 정점 번호의 최솟값이 $L^*(i)$, 최댓값에 1을 더한 값이 $R^*(i)$ 이다.

구간으로 향하는 간선들은 세그먼트 트리 모양으로 희소화할 수 있다. 세그먼트 트리의 각 내부 정점에서 두 자식으로 간선을 연결해 두면, $i \rightarrow [f(i), g(i+1) - 1]$ 꼴의 구간 간선은 구간을 덮는 $O(\log N)$ 개의 트리 정점으로서의 간선으로 대체된다. 정점 $O(N)$ 개, 간선 $O(N \log N)$ 개의 그래프를 얻고, 부분문제 3과 같이 강한 연결 요소로 압축한 뒤 위상 정렬 역순으로 도달 가능한 단위 구간 번호의 최솟값과 최댓값을 전파하면 모든 $L^*(i), R^*(i)$ 를 $O(N \log N)$ 에 얻는다.

전체 시간복잡도는 $O(N \log N + M + Q)$ 이다.

공장 - 풀이

작성자: 송준혁

관찰. 밤 d 의 근무자는 날짜 d 와 $d+1$ 의 근무자를 합친 것이므로, 모든 시간대의 인원이 짝수일 필요충분조건은 각 날짜 d 에 대해 $D_i = d$ 인 고용 인원이 짝수인 것이다.

부분문제 1

2^N 개의 부분집합 각각에 대해 관찰의 조건을 확인하고, 각 시간대의 근무자를 정렬해 값을 직접 계산한다. 시간복잡도는 $O(2^N \cdot N \log N)$ 이다.

부분문제 2

모든 지원자가 1일차에 근무하므로, 고용 집합을 A 오름차순으로 정렬하면 인접한 두 명씩 짝이 되고 낮과 두 밤(0일차, 1일차)의 짝이 모두 같다. 따라서 짝 (x, y) ($A_x < A_y$)의 기여는 $(B_y - B_x) - (C_x + C_y) - 2(A_y - A_x)$ 로 독립적이다.

지원자를 A 오름차순으로 훑으며 $dp[j][p] = (j\text{번째 지원자까지 결정했고 아직 닫히지 않은 짝이 } p \in \{0, 1\} \text{ 개일 때의 최댓값})$ 을 계산한다. j 번째 지원자를 짝의 아래 사람으로 고용하면 $-B_j - C_j + 2A_j$, 위 사람으로 고용하면 $+B_j - C_j - 2A_j$ 를 더한다. 답은 $dp[N][0]$ 이고, 시간복잡도는 $O(N \log N)$ 이다.

부분문제 3

전체 지원자를 A 오름차순으로 훑는 DP를 하자. 상태는 각 날짜별로 지금까지 고용한 인원의 기우성(닫히지 않은 짝의 존재 여부)을 담은 마스크 m (2^T 가지)이다.

짝이 정렬 순서에서 인접한 두 명씩이므로, 밤의 수당은 인접한 두 A 값 사이의 길이 g 인 구간마다 그 아래의 해당 밤 근무자가 홀수 명일 때 g 씩 발생한다. 밤 d 의 기우성은 날짜 d 와 $d+1$ 의 기우성의 XOR이므로, 홀수인 밤의 개수는 $f(m) = \text{popcount}(m \oplus 2m)$ 이다. 인접한 지원자 사이의 A 간격 g 마다 $g \cdot f(m)$ 을 빼고, 지원자 j (날짜 d)를 고용하면 비트 d 를 토글하며 짝을 열 때 $-B_j - C_j$, 닫을 때 $+B_j - C_j$ 를 더한다. 답은 $dp[N][0]$ 이고, 시간복잡도는 $O(2^T N)$ 이다.

이 DP는 부분문제 5의 다이어그램을 아래에서 위로 스윕핑하며 각 열이 현재 영역에 포함되는지를 관리하는 것과 같다.

부분문제 4

날짜 순서로 DP를 하자. $dp[d][S] = (\text{날짜 } 1, \dots, d \text{의 고용을 정했고 날짜 } d \text{의 고용 집합이 } S \text{이며 } d-1 \text{일차 밤까지 정산했을 때의 최댓값})$ 으로 정의하면, 이전 날짜의 집합 S' 을 열거해

$$dp[d][S] = \max_{S'} (dp[d-1][S'] + \text{day}(d, S) - \text{night}(d-1, S', S))$$

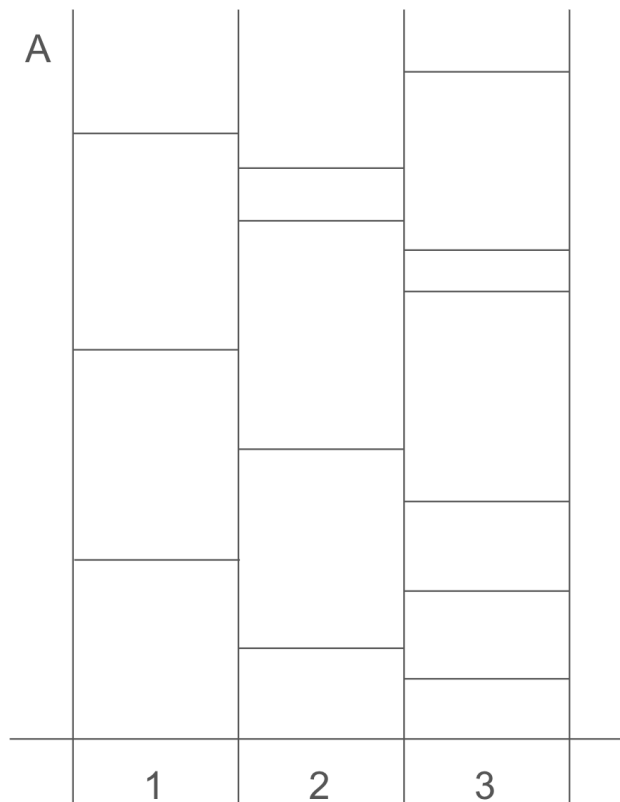
이다. $\text{day}(d, S)$ 는 $|S|$ 가 홀수면 $-\infty$, 짝수면 (낮 이익) $-$ (기본 임금 합)이고, **night**는 두 집합을 병합 정렬해 짝별 차의 합으로 직접 계산한다. T 일차 밤은 빈 날짜 $T + 1$ 을 추가해 처리한다.

날짜별 인원 수를 $k_d \leq 8$ 이라 하면 시간복잡도는 $O(\sum_d 2^{k_{d-1}+k_d}(k_{d-1} + k_d))$ 로 충분히 작다.

이 DP는 부분문제 5의 다이어그램을 왼쪽에서 오른쪽으로 스위핑하며 관리하는 것과 같다.

부분문제 5

다이어그램. 가로축에 날짜, 세로축에 누적 근무 시간을 놓고, 지원자 i 를 열 D_i 의 높이 A_i 에 걸친 가로 선분으로 그리자. 각 열은 자신의 선분들에 의해 위아래로 쌓인 칸들로 분할된다.



영역과의 대응. 관찰의 조건을 만족하는 고용 집합은 칸들의 합집합인 **영역**과 일대일로 대응된다. 날짜 d 에서 고용자들이 이루는 각 짝 (x, y) 를 열 d 의 구간 (A_x, A_y) 로 바꾸면 이 구간들은 서로소이고, 그 합집합이 열 d 의 영역이다. 역으로 임의의 영역에서 각 열의 극대 구간의 위아래 끝은 그 열의 선분이므로, 경계에 놓인 선분의 지원자들을 고용하면 날짜별 인원이 짝수인 고용 집합을 얻는다. (지원자가 한 명뿐인 날짜는 칸을 만들지 않으므로, 어차피 고용될 수 없는 그 지원자는 자연히 배제된다.)

이 대응에서 문제의 값을 영역에 대한 값으로 나타내자. 먼저 경계에서 생기는 값은 다음과 같다.

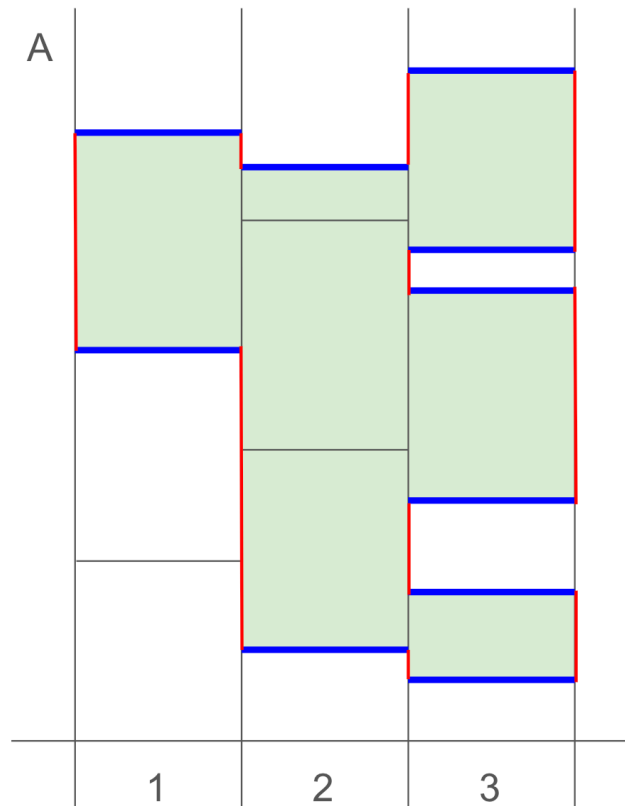
- 영역의 위쪽 경계 선분 i 는 짝의 위 사람이므로 $+B_i$, 아래쪽 경계 선분 i 는 $-B_i$ 를 기여하고, 경계 선분마다 기본 임금 C_i 를 지불한다. 경계가 아닌 선분(영역 내부를 지나거나 영역 밖)의 지원자는 고용되지 않는다.
- 밤 d 의 짝도 정렬 순서에서 인접한 두 명씩이므로, 수당은 열 d 와 $d + 1$ 사이 수직선 위에서 아래 인원의 기우성이 홀수인 높이의 총 길이이다. 이 기우성은 두 열의 기우성의 XOR이고 각 열의 기우성이 홀수인 높이가 정확히 그 열의 영역이므로, 수당은 두 열의 영역의 대칭차의 길이, 즉 그 수직선 위에 드러난 영역의 세로 경계의 길이이다. 0일차 밤과 T 일차 밤은 빈 열과의 경계인 왼쪽·오른쪽 바깥벽에 해당한다.

여기서 B 기여는 경계가 아니라 면적에 부여할 수 있다. 아래 선분이 u , 위 선분이 v 인 칸 f 에 가치 $v_f = B_v - B_u$ 를 부여하자. 영역의 각 극대 구간에서 (위 경계의 B) - (아래 경계의 B)는 사이 선분들의 B 가 연속적으로 소거되어 내부 칸들의 v_f 합과 같으므로, 영역 R 전체의 B 기여는 $\sum_{f \in R} v_f$ 이다.

따라서 문제는 평면 분할에서 칸들의 합집합인 영역 R 을 골라

$$\underbrace{\sum_{f \in R} v_f}_{\text{면적의 가치}} - \underbrace{\left(\sum_{\text{경계 가로 선분 } i} C_i + \text{세로 경계의 총 길이} \right)}_{\text{둘레의 비용}}$$

을 최대화하는 문제, 즉 면적의 가치와 둘레의 비용의 차를 최대화하는 영역 선택 문제가 된다. 아래 그림에서 영역의 각 칸마다 v_f 를 얻고, 둘레 중 파란 가로 경계마다 C 를, 빨간 세로 경계의 길이만큼 수당을 지불한다.



최소 컷 모델링. 이러한 (면적의 가치) - (둘레의 비용) 형태의 목적 함수는 최소 컷으로 최적화할 수 있다. 각 칸을 정점으로 하고 소스 s 와 싱크 t 를 추가한 그래프를 만들자. 이 그래프를 s 와 t 가 분리되도록 자르면 모든 칸이 s 측과 t 측으로 나뉘는데, s 측에 붙은 칸은 영역으로 고르는 면, t 측에 붙은 칸은 고르지 않는 면으로 간주한다. 각 열의 최하단 아래와 최상단 위의 바깥은 항상 고르지 않으므로 t 와 동일시한다. 이제 어떤 컷이든 그 용량이 해당 영역 선택의 손실과 일치하도록 다음과 같이 간선을 짓는다.

- $\text{base} = \sum_f \max(v_f, 0)$ 에서 시작해, $v_f > 0$ 이면 $s \rightarrow f$ 에 용량 v_f (f 를 선택하지 않으면 손실), $v_f < 0$ 이면 $f \rightarrow t$ 에 용량 $-v_f$ (f 를 선택하면 손실)의 간선을 짓는다.
- 각 선분 i 에 대해, i 를 사이에 둔 위아래 두 칸 사이에 용량 C_i 의 양방향 간선을 짓는다. 두 칸의 선택이 다를 때, 즉 i 가 둘레의 가로 경계일 때 절단된다.
- 각 행 d ($0 \leq d \leq T$)에 대해, 열 d 와 $d+1$ 사이 수직선을 두 열의 모든 선분 높이로 잘라 기본 구간들로 나누고, 각 기본 구간의 좌우에 인접한 두 칸 사이에 용량이 구간의 길이인 양방향 간선을 짓는다. 좌우의 선택이 다를 때, 즉 세로 경계가 드러날 때 그 길이만큼 절단된다.

바깥이 t 측에 고정되어 있으므로 s 측 칸 집합과 영역 R 이 일대일 대응되고, 절단 용량은 정확히 $\text{base} - (\text{목적 함수})$ 이다. 따라서 답은 $\text{base} - (\text{최소 절단})$ 이다. 최대 유량을 흘린 뒤 잔여 그래프에서 s 로부터 도달 가능한 칸들이 최적 영역이고, 선택된 칸에 흘수 번(정확히 한 번) 인접한 선분의 지원자들을 고용하면 된다.

칸은 $O(N)$ 개, 간선도 각 수직선의 기본 구간 수의 합이 $O(N)$ 이므로 $O(N)$ 개이고, 최대 유량은 $\text{base} = O(N \max B)$ 이하이다. 따라서 Ford-Fulkerson으로도 시간복잡도 $O(E \cdot F) = O(N^2 \max B)$ 로 충분하다.